

Contenido:

- Estructuras de control en guiones.
- `if`, `case`
- `for`, `while`, `until`
- `break`, `continue`

Consejos:

- Use el tabulador para completar las rutas
- Compruebe el resultado de las órdenes que ejecute
- Use `man` para obtener ayuda de las distintas órdenes

Enunciado:

1. Muestre por pantalla el texto “Practica 6: Scripts (Estructuras de Control) – Inicio...”

```
echo "Practica 6: Scripts (Estructuras de Control) – Inicio..."
```

2. Cree un directorio llamado `Practica6` que sea hijo de `ModuloI`. Sitúese dentro del directorio recién creado.

```
mkdir ./ModuloI/Practica6
```

```
Vamos al directorio recién creado
```

```
cd ./ModuloI/Practica6
```

3. En el directorio `Practica6` cree un script llamado `miron`. Este script debe recoger un único parámetro y realizar las siguientes acciones.

Si no se pasa un parámetro se visualizará un mensaje de error y finalizará.

Si se trata de un directorio, debemos ver el contenido del mismo con el comando `ls -la`. Elimine la posible salida de errores de la pantalla.

Si el parámetro es un enlace simbólico, debe visualizar su contenido con el comando `cat`. Previamente a la salida, debe de mostrar por pantalla la ruta al fichero original el cual enlaza.

Si el parámetro es un fichero ordinario debe visualizar su contenido con el comando `more`.

Si no es ni un fichero, ni enlace, ni directorio debe mostrar un mensaje de error junto con el valor del parámetro.

```
joe nprimeros.sh
```

```
-----
```

```
#!/bin/bash
```

```
if [ $# -ne 1 ]; then
```

```
    echo "Error: el script esperaba un parametro!"
```

```
    exit 10
```

```
fi
```

```
if [ -d $1 ]; then
    ls -la $1 2>/dev/null
elif [ -L $1 ]; then
    echo "Enlace simbolico a: $1"
    cat $1
elif [ -f $1 ]; then
    more $1
else
    echo "Error: parametro $1 desconocido!"
    exit 11
fi

-----

chmod a+x miron.sh (Esto sólo se ejecutaría la primera vez, para dar los
permisos)

./miron.sh ~
```

4. Dentro del directorio `Practica6` cree un script llamado `nprimeros.sh`, que tome dos parámetros. El primer parámetro deberá ser un directorio, y el segundo un número. El script deberá realizar las siguientes acciones:

Si el primer parámetro no es un directorio, se dará un mensaje de error y finalizará el script

Si el primer parámetro es un directorio válido, visualizará los primeros `n` ficheros del listado del directorio (donde `n` se corresponde con el segundo parámetro) con el siguiente mensaje:

El nombre del fichero i es <<nombre del fichero>>.

Donde `i` es el orden del fichero en el listado.

NOTA: El número `i` se implementa con un contador que va incrementándose en un bucle controlado por los ficheros a visualizar.

```
joe nprimeros.sh

-----

#!/bin/bash

itI=1
nMax=$2

if [ ! -d $1 ]; then
    echo "Error: el primer parametro debe ser un directorio valido!"
    exit 10
```

```

else
    for i in $(find $1 -maxdepth 1 -type f -printf "%f\n")
    do
        if [ $itI -gt $nMax ]; then
            break
        fi

        echo "El nombre del fichero $itI es $i."

        itI=$(expr $itI + 1)
    done
fi
-----

chmod a+x nprimeros.sh (Esto sólo se ejecutaría la primera vez, para dar
los permisos)

./nprimeros.sh ~ 3

NOTA: se recomienda en los scripts (para este y en adelante), realizar
todas las pruebas necesarias para controlar las diferentes salidas que
pueda tener el script. Por ejemplo, probar a lanzar el script con un
primer parámetro de un directorio no válido.

```

5. Cree dos directorios llamados `largos` y `cortos` dentro del directorio `Practica6`.

```
mkdir largos cortos
```

6. Cree, dentro de `Practica6`, un script llamado `selector.sh` que puede tomar una secuencia de parámetros de cualquier longitud. Para cada parámetro se comprobará si se corresponde con un fichero del directorio de usuario, y de ser así lo copia al directorio `largos` si su nombre tiene más de 6 caracteres y al directorio `cortos`, si su nombre tiene seis o menos caracteres.

NOTAS:

- Se deberá usar un `case` dentro de un bucle `for`.
- Comprobar qué es lo que haría cuando se trata de un enlace simbólico a un fichero de texto

```

joe selector.sh
-----

#!/bin/bash

for i in $*

```

```
do
    if [ -f ~/$i ]
    then
        #echo "$i es un fichero"
        case $i in
            ??????*) cp ~/$i largos
                ;;
            *) cp ~/$i cortos
                ;;
        esac
    fi
done
-----
chmod a+x selector.sh (Esto sólo se ejecutaría la primera vez, para dar
los permisos)
./selector.sh ~ 3 prueba.txt pruebaLink

NOTA: cuando se copia un enlace simbólico, lo que hace sería copiar con
el nombre del enlace, el fichero al cual enlaza con los permisos que
tuviera inicialmente dicho fichero.
```

7. Cree un script en el directorio `Practica6` llamado `replica.sh`, que tome un parámetro y haga lo siguiente:

Si no se pasa un parámetro se visualizará un mensaje de error y finalizará.

Si tenemos parámetro, pedirá que se introduzca un número por el teclado entre 2 y 9.

Si no es inferior a 10, mostrará un mensaje por pantalla y ejecutará el script con el mayor valor posible, esto es con 9. Lo mismo haría con el mínimo, que sería 2.

A partir de aquí, el script creará tantos directorios anidados, con el nombre que se ha pasado como primer parámetro, como indique el número leído.

Ejemplo: se ejecuta `replica p1`, y el número que se lee es 3. Esto creará un directorio `p1`, que tiene dentro un directorio `p1`, que a su vez tiene dentro un directorio `p1`.

NOTA: se monta un bucle `while`, que itera tantas veces como indique el contador, y dentro se va creando la ruta de los subdirectorios sobre una variable, que es la usada para crear los subdirectorios.

```
joe replica.sh
-----

#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Error: se esperaba un parametro!"
    exit 10
else
    echo "Por favor, introduzca un numero entre 2 y 9:"
    read VAL

    if [ $VAL -gt 9 ]; then
        echo "Valor superior excedido, tomaremos el valor maximo,
9."
        VAL=9
    elif [ $VAL -lt 2 ]; then
        echo "Valor minimo excedido, tomaremos el valor minimo,
2."
        VAL=2
    fi

    contador=0
    base="."
    while [ $contador -lt $VAL ]
    do
        echo "Creando directorio $base/$1"
        mkdir $base/$1
        contador=$(expr $contador + 1)
        base=$base/$1
    done

fi

-----

chmod a+x replica.sh (Esto sólo se ejecutaría la primera vez, para dar
los permisos)

./replica.sh prueba.txt
```

8. En el directorio `Practica6`, cree un script llamado `verdire.sh`. Este script deberá mostrar de forma continuada el contenido de un directorio. Este script podrá recoger un parámetro.

Si se ha pasado el parámetro y se corresponde a un directorio válido, mostrará el contenido de dicho directorio cada segundo de forma continuada.

Si el parámetro pasado no se corresponde con un directorio válido, dará un mensaje de error y finalizará.

Si no se pasa ningún parámetro, se deberá visualizar el directorio de trabajo.

El script no finalizará nunca, es decir, se deberá 'romper su ejecución' con CTRL+C cuando se quiera finalizar.

```
joe verdire.sh

-----

#!/bin/bash

DIR=$1

if [ $# -eq 0 ]; then
    DIR=~
fi

if [ -d $DIR ]; then
    cont=1
    while [ $cont -eq 1 ]
    do
        ls -ls $DIR
        sleep 1
    done
else
    echo "Error: $DIR no es un directorio valido!"
    exit 10
fi

-----

chmod a+x verdire.sh (Esto sólo se ejecutaría la primera vez, para dar los permisos)

./verdire.sh
```

9. Cree un script llamado `reubicadoSelectivo.sh` en el directorio `Practica6`. El script funcionará de la siguiente forma:

Si el número de parámetros no es igual a 2 mostrará el mensaje "Número de parámetros incorrecto, se esperaban 2" y finalizará con un código de estado resultante de 11.

Si los dos parámetros no son directorios válidos mostrará el mensaje "Los parámetros deben ser directorios válidos" y finalizará con un código de estado resultante de 21.

El programa solicitará al usuario que introduzca por teclado un carácter o número.

Por cada fichero ordinario que exista en el directorio que se pasa como primer parámetro, que comience por el carácter o número introducido en el paso anterior, se mostrará su nombre y sus 3 últimas líneas, y se preguntará si se quiere mover. Si la respuesta del usuario comienza por 's', el fichero se moverá al directorio pasado como segundo parámetro. Si no, no se realizará nada con él.

Al finalizar el script, se informará de la cantidad de ficheros que han sido movidos.

```
joe reubicadoSelectivo.sh
```

```
-----  
#!/bin/bash  
  
if [ $# -ne 2 ]; then  
    echo Numero de parametros incorrecto, se esperaban 2  
    exit 11  
elif [ ! -d $1 -o ! -d $2 ]; then  
    echo Los parametros deben ser directorios validos  
    exit 21  
else  
    echo "Por favor, introduzca un caracter o numero:"  
    read ini  
    total=0  
    for i in $(find $1 -maxdepth 1 -name "$ini*" -type f)  
    do  
        echo $i  
        tail -3 $i  
        echo  
        echo "¿Quiere moverlo?(s/n)"  
        read respuesta  
        case $respuesta in
```

```
s*) echo Se copia
    mv $i $2
    total=$((total+1))
    ;;
*) echo Se omite

esac

echo

done

echo Se han movido $total ficheros
fi

-----

chmod a+x reubicadoSelectivo.sh (Esto sólo se ejecutaría la primera vez,
para dar los permisos)

./reubicadoSelectivo.sh ~ largos
```

10. Cree un script en el que realice pruebas de ejecución de `break` y `continue` para las tres estructuras iterativas vistas en la lección (`for`, `while` y `until`). Para ello, crearemos un script llamado `controlBucle.sh` que esperará un parámetro que indique una de las tres instrucciones anteriores. Los tres métodos realizarán lo mismo:

Mediante un bucle entre uno y diez, el script mostrará por pantalla el valor del índice del bucle actual salvo que sea el 6, que no mostrará nada por pantalla y pasará a la siguiente iteración.

En el caso que se llegue a la penúltima iteración, no se mostrará por pantalla dicha iteración y el script deberá saltar al final del bucle obviando las iteraciones que quedaran.

NOTA: usar obligatoriamente `break` y `continue` para resolver dichas cuestiones

```
joe controlBucle.sh

-----

#!/bin/bash

if [ $# -ne 1 ]; then
    echo Numero de parametros incorrecto, se esperaba 1
    exit 11
else
    if [ $1 == "for" ]; then
        for i in {1..10}
        do
```



```
        if [ $i -eq 6 ]; then
            continue
        elif [ $i -eq 9 ]; then
            break
        fi
        echo "Iteracion numero: $i"
    done
elif [ $1 == "while" ]; then
    i=1
    while [ $i -le 10 ]
    do
        if [ $i -eq 6 ]; then
            i=$(expr $i + 1)
            continue
        elif [ $i -eq 9 ]; then
            break
        fi
        echo "Iteracion numero: $i"
        i=$(expr $i + 1)
    done
elif [ $1 == "until" ]; then
    i=1
    until [ $i -gt 10 ]
    do
        if [ $i -eq 6 ]; then
            i=$(expr $i + 1)
            continue
        elif [ $i -eq 9 ]; then
            break
        fi
        echo "Iteracion numero: $i"
        i=$(expr $i + 1)
    done
else
    echo "Error: el parametro debe contener [for, while, until]!"
```

```
        exit 11

    fi

fi

-----

chmod a+x controlBucle.sh (Esto sólo se ejecutaría la primera vez, para
dar los permisos)

./controlBucle.sh for
```

11. Muestre en pantalla el mensaje “Práctica 6: Scripts (Estructuras de Control) ... Finalizada”

```
echo "Practica 6: Scripts (Estructuras de Control) ... Finalizada"
```

12. Salga adecuadamente del sistema

```
exit
```